

بخش اول

مقدمه

در این تمرین به دنبال پیاده سازی الگوریتم های متفاوت برای بازیابی سیگنال های تُنک هستیم. در ابتدا، الگوریتم Subset Selection را پیاده سازی می کنیم. در ادامه، نُرم دو سیگنال را صفر می کنیم و بررسی می کنیم که آیا انجام این کار کمکی به بازیابی سیگنال می کند یا خیر.

در ادامه، به دنبال پیاده سازی الگوریتم Matching Pursuit (MP) هستیم. همچنین، می کشیم تا بدون دانستن N_0 نیز به حل مسئله پردازیم. سپس، روشی مشابه اما بهتر را به کار می گیریم. این الگوریتم Orthogonal Matching Pursuit (OMP) نام دارد و تفاوت آن با الگوریتم MP این است که بعد از در روش OMP، بعد از پیدا کردن ماکزیمم، برای S_i های جدید، قبلی ها را جوری به روز می کنیم که منابع بهینه داشته باشیم. سپس، اقدام به پیاده سازی الگوریتم Basis Pursuit (BP) می کنیم و بررسی می کنیم که دانستن N_0 برای حل این قسمت ضروری است یا خیر.

در انتها، الگوریتم Iteratively Reweighted Least Square (IRLS) را پیاده سازی می کنیم و مجدداً بررسی می کنیم که دانستن N_0 برای حل این قسمت ضروری است یا نه. همچنین برای جمع بندی الگوریتم های این تمرین، به دنبال پیدا کردن بهترین الگوریتم هستیم و برای این انتخاب معیار های خود را بیان می کنیم.

قسمت الف

برای این قسمت ابتدا تعداد تمام حالاتد ممکن را بدست می آوریم. می دانیم که تعداد تمام این حالات برابر با $\binom{N}{N_0}$ می باشد. (تعداد N برابر با 60 و تعداد N_0 برابر با 3 می باشد). در نتیجه تعداد تمام حالات برابر با 34220 می باشد.

حال عبارت tic را تایپ می کنیم تا متلب شروع به محاسبه زمان اجرا کند. سپس توسط تابع `nchoosek()` تمام حالاتی که ممکن است پیش بیاید را بدست می آوریم. سپس یک تانسور به نام `D_hat` تعریف می کنیم و تمام حالات ممکن را توسط یک `for` در آن می ریزیم. در ادامه در این `for` هر `s_hat` را که بدست آوردیم، متناظر با آن خطا (`err`) آن را بدست می آوریم. سپس بررسی می کنیم که آیا مقدار این خطا از `err` ای که تعیین کردیم کمتر است یا خیر و اگر این مقدار کمتر بود، آن را داخل یک متغیر دیگر می ریزیم. در انتها، به نتایج زیر می رسم:

• مقدار زمان اجرای الگوریتم: 1.084438 [s]

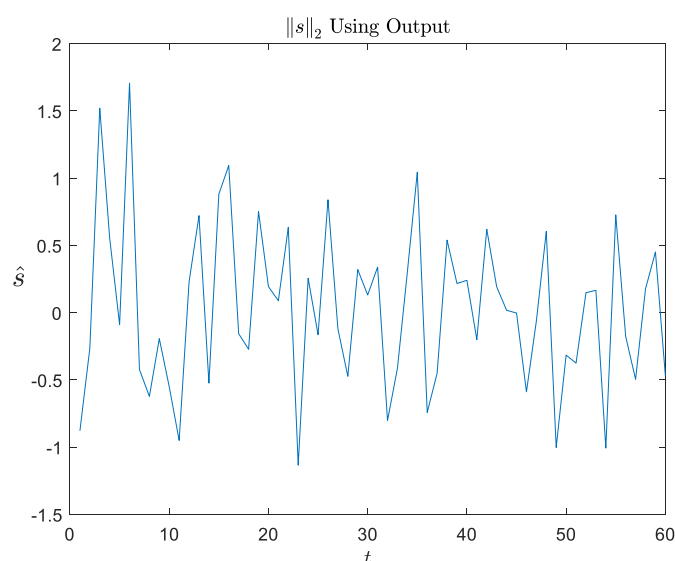
$$\hat{S} = \begin{bmatrix} 5 \\ 7 \\ -3 \end{bmatrix} \cdot$$

• مقدار خطا: 6.7605×10^{-16}

مشاهده می شود که این روش زمان زیادی برای بدست آوردن جواب نیاز دارد. دانستن N_0 نیز کمک زیادی به مسئله می کند زیرا اگر اطلاعاتی از N_0 نداشته باشیم، باید از $N_0 = 1$ شروع به اجرای الگوریتم بکنیم تا جایی که خطای ما از یک مقدار معین کم تر شود. در نتیجه دانستن N_0 در جهت کاهش سرعت کمک زیادی می کند.

قسمت ب

می دانیم که اگر به جای $\|s\|_0$ از $\|s\|_2$ استفاده کنیم، ماتریس تقریب منابع به صورت $\hat{S} = D^\dagger x$ می باشد. حال با اجرای این کد و مشاهده نمودار به این نتیجه می رسیم که ماتریس بدست آمده، Sparse نمی شود، در نتیجه استفاده از $\|s\|_2$ به جای $\|s\|_0$ ایده خوبی نمی باشد. همچنین، دانستن N_0 کمکی به حل این مسئله نمی کند. نمودار گفته شده در تصویر ۱ آمده است.



تصویر ۱: تقریب بدست آمده از s هنگامی که از $\|s\|_2$ به جای $\|s\|_0$ استفاده می کنیم

بخش ج

برای این قسمت ابتدا ضرب داخلی x با تمام d_i ها را بدست می آوریم و d_j را بدست می آوریم که آن برابر با اتم با بیشترین تصویر روی x می باشد. سپس، اثر این اتم را از روی x کم کرده و دوباره این کار را انجام می دهیم. چون از گفته های سوال تعداد درایه های غیر صفر مسئله را می دانیم، یک حلقه for به تعداد N_0 می زنیم و این الگوریتم را به این اندازه اجرا می کنیم. با اجرای این الگوریتم و با دانستن $N_0 = 3$ ، به مقادیر صفحه زیر می رسیم:

• مقدار زمان اجرای الگوریتم: $0.003628 [s]$

$$\hat{S} = \begin{bmatrix} 8.870176 \\ 4.840268 \\ 1.711150 \end{bmatrix} \bullet$$

• مقدار خطا: 0.16015

می دانیم که داشتن N_0 به افزایش سرعت اجرای کد کمک زیادی می کند. حال فرض می کنیم که N_0 را نداریم. بنابراین در یک حلقه **for** از $N_0 = 1$ شروع به اجرای کد می کنیم تا جایی که خطای ما از مقدار **err_thr** کمتر شود (مقدار **err_thr** را 10^{-1} فرض کرده ایم). البته یک روش بهتر این است که فرض $N_0 = 10$ را بگیریم و ماتریس $\hat{D}$ ای را بدست آوریم که تمام اتم ها را شامل شود. در حلقه **for**، هر بار **err** را محاسبه می کنیم و در انتها اندیسی را پیدا می کنیم که **err** در آنجا از **err_thr** کمتر باشد.

با اجرای این کد به نتایج زیر می رسیم:

• مقدار زمان اجرای الگوریتم: 0.006297 [s]

$$N_0 = 5 \bullet$$

• مقدار خطا: 0.091557

در نتیجه دانستن N_0 در این قسمت علاوه بر افزایش سرعت، تعیین می کند که به چه تعداد اتم نیاز داریم. از الگوریتم **Subset Selection** می دانیم که با $N_0 = 3$ می توان اتم هایی را انتخاب کرد که خطا را صفر می کنند. اما در این الگوریتم اگر N_0 را نداشته باشیم در $N_0 = 5$ خطای ما از $\text{err_thr}(10^{-1})$ کمتر می شود.

قسمت د

این روش نیز مشابه روش **MP** (قسمت ج) می باشد؛ با این تفاوت که در حلقه **for** این روش از مرتبه دوم به بعد اقدام به بهینه سازی می کنیم تا اتم های باقیمانده بهتر انتخاب شوند. با اجرای این الگوریتم و با دانستن $N_0 = 3$ ، به مقادیر صفحه زیر می رسیم:

• مقدار زمان اجرای الگوریتم: 0.003372 [s]

$$\hat{S} = \begin{bmatrix} 8.045944 \\ 4.837252 \\ 1.881823 \end{bmatrix} \bullet$$

• مقدار خطا: 0.15235

می دانیم که داشتن N_0 به افزایش سرعت اجرای کد کمک زیادی می کند. حال فرض می کنیم که N_0 را نداریم. بنابراین در یک حلقه **for** از $N_0 = 1$ شروع به اجرای کد می کنیم تا جایی که خطای ما از مقدار **err_thr** کمتر شود (مقدار **err_thr** را 10^{-1} فرض کرده ایم). البته یک روش بهتر این است که فرض $N_0 = 10$ را بگیریم و ماتریس $\hat{D}$ ای را

بدست آوریم که تمام اتم‌ها را شامل شود. در حلقه `for`، هر بار `err` را محاسبه می‌کنیم و در انتها اندیسی را پیدا می‌کنیم که `err` در آنجا از `err_thr` کمتر باشد.

با اجرای کد توضیح داده شده در بالا، به نتایج زیر می‌رسیم:

- مقدار زمان اجرای الگوریتم: $0.007541 [s]$

- $N_0 = 5$

- مقدار خطا: 0.072322

در نتیجه دانستن N_0 در این قسمت علاوه بر افزایش سرعت، تعیین می‌کند که چه تعداد اتم نیاز داریم. از الگوریتم Subset Selection می‌دانیم که با $N_0 = 3$ می‌توان اتم‌هایی را انتخاب کرد که خطا را صفر می‌کنند. اما در این الگوریتم اگر N_0 را نداشته باشیم در $N_0 = 5$ خطای ما از $err_thr(10^{-1})$ کمتر می‌شود. در مقایسه با الگوریتم OP می‌توانیم `err_thr` را کمتر کنیم تا حدی که یک الگوریتم قابل به کم کردن خطا تا آن حد نباشد. برای این کار، `err_thr` را برابر 10^{-2} قرار می‌دهیم و مجدداً نتایج را بررسی می‌کنیم.

الگوریتم MP :

- مقدار زمان اجرای الگوریتم: $0.006255 [s]$

- $N_0 = 10$

- مقدار خطا: 0.002424

الگوریتم OMP :

- مقدار زمان اجرای الگوریتم: $0.007676 [s]$

- $N_0 = 8$

- مقدار خطا: 0.0048044

مشاهده می‌شود که در الگوریتم MP با این که از تمام اتم‌ها (10) استفاده کرده ایم، نتوانسته ایم که خطا را کمتر از خطای مطلوبمان کنیم؛ در صورتی که در الگوریتم OMP با تعداد اتم کمتر (8) توانسته ایم به خطای مورد نظر برسیم.

بخش ه

الگوریتم این بخش به صورت ریاضی در زیر آمده است:

$$\begin{cases} \hat{s} = \underset{s}{\operatorname{argmin}} \|s\|_1 = \sum_{n=1}^N |s_n| \xrightarrow{s_n = s_n^+ + s_n^-} = \sum_{n=1}^N s_n^+ + s_n^- \\ s.t. \ x = Ds \rightarrow x = [D \quad -D] \begin{bmatrix} s^+ \\ s^- \end{bmatrix} \xrightarrow{[D \quad -D] = y, \begin{bmatrix} s^+ \\ s^- \end{bmatrix} = y} x = D_1 y \end{cases} \Rightarrow \begin{cases} s_n^+ = \max(s_n, 0) \\ s_n^- = \max(-s_n, 0) \end{cases}$$

در انتها فرم این مسئله به صورت زیر در می آید:

$$\begin{cases} y = \underset{y}{\operatorname{argmin}} \sum_{i=1}^{2N} y_i \\ s.t. \ x = D_1 y, \ y_i > 0 \end{cases} \rightarrow \text{Linear Programming}$$

در نتیجه این قسمت با تابع داخلی `linprog()` بدست می آید. پس از محاسبه، نتایج این الگوریتم به صورت زیر می باشد:

• مقدار زمان اجرای الگوریتم: $0.009908 [s]$

$$\hat{s} = \begin{bmatrix} 5 \\ 7 \\ -3 \end{bmatrix}$$

• مقدار خطا: 2.3272×10^{-16}

همینطور که در این قسمت می بینیم، هیچ استفاده ای از N_0 نکردیم و دانستن یا ندانستن N_0 در این الگوریتم کمک چندانی به ما نمی کند.

بخش و

الگوریتم این بخش به صورت ریاضی در زیر آمده است:

$$\begin{cases} \hat{s} = \underset{s}{\operatorname{argmin}} \|s\|_1 = \sum_{n=1}^N |s_n| \xrightarrow{app} = \sum_{n=1}^N \omega_n |s_n|^2 \rightarrow \begin{cases} f(s) = s^T W s \\ s.t. \ x = Ds \end{cases} \\ s.t. \ x = Ds \end{cases} \rightarrow \begin{cases} \hat{s} = \frac{1}{2} W^{-1} D^T \lambda \\ \lambda = 2 (D W^{-1} D^T)^{-1} x \end{cases} \Rightarrow \hat{s} = W^{-1} D^T (D W^{-1} D^T)^{-1} x$$

این روش به صورت **iterative** می باشد یعنی باید چندین بار اجرا شود، بنابراین برای پیاده سازی آن از یک حلقه **for** استفاده می کنیم. همچنین، اگر قیدی روی عبارت های ماتریس W نگذاریم، به علت بسیار کوچک شدن درایه های

ماتریس، ماتریس به تکین شدن متمایل می‌شود؛ بنابراین برای جلوگیری از اختار، روی مقادیر بسیار بزرگ و بسیار کوچک ماتریس یک قید اعمال می‌کنیم. نتیجه این الگوریتم برای $\text{iterNum} = 100$ به صورت زیر می‌باشد:

• مقدار زمان اجرای الگوریتم: $0.011046 [s]$

$$\hat{S} = \begin{bmatrix} 5 \\ 7 \\ -3 \end{bmatrix} \cdot$$

• مقدار خطا: 9.7861×10^{-17}

همینطور که در این قسمت می‌بینیم، هیچ استفاده‌ای از N_0 نکردیم و دانستن یا ندانستن N_0 در این الگوریتم کمک چندانی به ما نمی‌کند.

بخش ی

اگر مقایسه ما بر اساس مقدار خطا باشد، روش **subset selection**، **BP** و **IRLS** کمترین خطا را دارند (خطایی تقریباً صفر). اما می‌دانیم که روش **subset selction** روشی با سرعت پایین می‌باشد و برای N_0 های بزرگ و یا مسئله‌هایی که N_0 آن‌ها نامعلوم است مناسب نمی‌باشد، زیرا این الگوریتم بسیار الگوریتم کندی است. همچنین با بررسی روش کمینه کردن نرم ۲، متوجه می‌شویم که این روش اصلاً روش خوبی نیست (در مسئله داده شده منبع تخمین زده شده توسط این روش اسپارس نشد). روش‌های **MP** و **OMP** روش‌هایی سریع می‌باشند، اما به علت ماهیت **Greedy** بودن این دو الگوریتم، از نظر دقت به خوبی سرعت خود نیستند. در انتها دو الگوریتم **BP** و **IRLS** باقی می‌مانند که هر دو روش هم سرعت و هم دقت بسیار خوبی دارند. اما از نظر من روش **BP** روش بهتری است؛ زیرا هم سرعت بهتری دارد و نیز الگوریتمی **iterative** نیست که نیاز به یک حلقه **for** داشته باشد.

برای جمع بندی می‌توان گفت که الگوریتم‌های **BP** و **IRLS** هر دو روش‌هایی با سرعت و دقت بسیار خوبی می‌باشند؛ اما روش **BP** به علت سرعت بهتر و تکرار شونده نبودن الگوریتم آن (همچنین عدم نیاز به یک عدد برای تکرار شدن)، بهترین روش می‌باشد.